ieSControl Integration Guide for LuxScaleAI

Overview: What Each Tool Does

	LuxScaleAI (current)	ieSControl (new)
IES parser	`ies_parser.py` from `ies-render/module/` — old, basic	`ies_analyzer.py` — full LM-63 standard implementation
Lumen estimation	Uses IES header `lumens_per_lamp × multiplier` only — misses files with 0 or wrong headers	`estimate_lumens()` — integrates actual candela data using zonal flux method (IES LM-63 standard)
Beam angle	Per-slice crossing, takes minimum across H planes	Per-plane + global peak, both 50% (beam) and 10% (field)
Uniformity grid	✅ Already does point-by-point via `uniformity_calculator.py`	Same method, but with better candela interpolation
Polar curve	❌ Not available to user	✅ Full polar PNG via `plot_polar()`
Room simulation	❌ None	✅ `panorama` endpoint — real ray-cast room with IES-accurate illuminance
3D distribution	❌ None	✅ 3D candela sphere

What to Copy

Step 1 — Copy `ies_analyzer.py` into LuxScaleAI

Copy this file:

```
ieSControl/ies_analyzer.py  →  LuxScaleAI/luxscale/ies_analyzer.py
```

No changes needed. It is self-contained (uses numpy, matplotlib, scipy — already in your requirements).

Step 2 — Add `scipy` to requirements.txt

Open `LuxScaleAI/requirements.txt` and add if not present:

```
scipy>=1.9.0
Pillow>=9.0.0
```

What to Replace / Upgrade

Step 3 — Replace lumen calculation in `ies_fixture_params.py`

File: `LuxScaleAI/luxscale/ies_fixture_params.py`

Find the `ies_params_for_file()` function. Change this line:

```
# OLD — only uses header value, wrong when header says 0 or -1
lumens = float(d.lumens_per_lamp) * float(d.multiplier)
```

Replace with:

```
# NEW — use integrated zonal flux from actual candela data (LM-63 standard)
from luxscale.ies_analyzer import parse_ies_file as _ies_analyze_parse, estimate_lumens as _estimate_l

def ies_params_for_file(ies_path: str) -> Dict[str, Any]:
    if not os.path.isfile(ies_path):
        raise FileNotFoundError(ies_path)

    # Parse with the accurate analyzer
    ies = _ies_analyze_parse(ies_path)
    metrics = _compute_all_metrics(ies)

    # Use integrated lumens (zonal flux method) — falls back to header if needed
    integrated_lumens = metrics.get("total_lumens", 0)
    header_lumens = float(ies.lumens_per_lamp) * float(ies.multiplier) * float(ies.num_lamps)

    # Prefer integrated lumens; use header as fallback if integration gives 0
    lumens = integrated_lumens if integrated_lumens > 0 else max(header_lumens, 0)

    # Beam angles — use global peak method (more accurate than per-slice minimum)
    beam_50 = metrics.get("beam_angle")      # 50% threshold = standard beam angle
    field_10 = metrics.get("field_angle")    # 10% threshold = field angle

    # Per-H asymmetry check
```

```
    per_h = metrics.get("per_h", {})
    h_beams = [v["beam"] for v in per_h.values() if v["beam"] is not None]
    b_min = min(h_beams) if h_beams else beam_50
    b_max = max(h_beams) if h_beams else beam_50
    asymmetric = (b_min is not None and b_max is not None and (b_max - b_min) > 3.0)

    return {
        "ies_path": ies_path,
        "lumens_per_lamp": lumens / max(ies.num_lamps, 1),
        "lumens_total": lumens,
        "lumens_integrated": integrated_lumens,     # NEW — from candela data
        "lumens_header": header_lumens,             # NEW — from IES header
        "num_lamps": int(ies.num_lamps),
        "max_candela": float(ies.max_value),
        "shape": ies.shape,
        "opening_width_m": float(ies.width),
        "opening_length_m": float(ies.length),
        "opening_height_m": float(ies.height),
        "beam_angle_deg": beam_50,                  # UPGRADED — global peak method
        "beam_angle_deg_max": b_max,
        "beam_angle_deg_first_slice": beam_50,
        "field_angle_deg": field_10,                # NEW — 10% threshold
        "beam_angle_asymmetric": asymmetric,
        "beam_angle_ies_signed_vertical": False,
        "efficacy_pct": metrics.get("efficacy_approx"),  # NEW — LOR %
    }
```

Step 4 — Add new Flask routes to `ai_routes.py` (or create `ies_routes.py`)

Create new file: `LuxScaleAI/luxscale/ies_routes.py`

```
"""
IES visualization routes — powered by ies_analyzer.py (ieSControl engine).
Adds polar curve, panorama/room simulation, and accurate parameter extraction.

Register in app.py:
    from luxscale.ies_routes import register_ies_routes
    register_ies_routes(app)
"""
from __future__ import annotations

import io
import os
import base64
```

```python
import tempfile
import uuid
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

from flask import Blueprint, request, jsonify
from luxscale.app_logging import log_step, log_exception

ies_viz_bp = Blueprint("ies_viz", __name__)

# In-memory session store: sid → {ies, metrics, path}
_IES_SESSIONS: dict = {}


def register_ies_routes(app):
    app.register_blueprint(ies_viz_bp)


def _fig_to_b64(fig, dpi=130):
    buf = io.BytesIO()
    from ies_analyzer import DARK_BG
    fig.savefig(buf, format="png", dpi=dpi, facecolor=DARK_BG, bbox_inches="tight")
    buf.seek(0)
    plt.close(fig)
    return base64.b64encode(buf.read()).decode()


# ---------------------------------------------------------------------------
# POST /api/ies/load
# Load an IES file by path (from the fixture catalog) and return metrics
# ---------------------------------------------------------------------------

@ies_viz_bp.route("/api/ies/load", methods=["POST"])
def api_ies_load():
    """
    Load an IES file and compute accurate metrics.

    Body: {"ies_path": "absolute or relative path to .ies file"}
    OR:   {"luminaire_name": "SC Flood 200W", "power_w": 200}

    Returns: session_id + full metrics summary
    """
    from luxscale.ies_analyzer import parse_ies_file, compute_all_metrics, estimate_lumens
```

```
    data = request.get_json(silent=True) or {}

    ies_path = data.get("ies_path", "").strip()

    # Resolve from luminaire name + power if path not given
    if not ies_path and data.get("luminaire_name"):
        try:
            from luxscale.ies_fixture_params import resolve_ies_path
            ies_path = resolve_ies_path(data["luminaire_name"], float(data.get("power_w", 0))) or ""
        except Exception as e:
            return jsonify({"status": "error", "message": f"Could not resolve IES path: {e}"}), 400

    if not ies_path or not os.path.isfile(ies_path):
        return jsonify({"status": "error", "message": "IES file not found"}), 404

    try:
        ies = parse_ies_file(ies_path)
        metrics = compute_all_metrics(ies)
    except Exception as e:
        log_exception("api_ies_load", e)
        return jsonify({"status": "error", "message": str(e)}), 422

    sid = str(uuid.uuid4())[:8]
    _IES_SESSIONS[sid] = {"ies": ies, "metrics": metrics, "path": ies_path,
                          "filename": Path(ies_path).name}

    declared_lm = None
    if ies.lumens_per_lamp > 0 and ies.num_lamps > 0:
        declared_lm = ies.lumens_per_lamp * ies.num_lamps * ies.multiplier

    log_step("api_ies_load", "loaded", sid=sid, file=Path(ies_path).name)

    return jsonify({
        "status": "success",
        "session_id": sid,
        "filename": Path(ies_path).name,
        "summary": {
            "peak_candela":      ies.max_value,
            "beam_angle":        metrics.get("beam_angle"),
            "field_angle":       metrics.get("field_angle"),
            "total_lumens":      metrics.get("total_lumens"),   # integrated (accurate)
            "declared_lumens":   declared_lm,                   # from header
            "lor_pct":           round(metrics.get("total_lumens", 0) / max(declared_lm or 1, 1) * 100
                                 if declared_lm and declared_lm > 0 else None,
            "num_lamps":         ies.num_lamps,
            "lumens_per_lamp":   ies.lumens_per_lamp,
            "shape":             ies.shape,
```

```
            "symmetry":          ies.symmetry_label(),
            "vertical_span":     ies.vertical_span_label(),
            "num_vertical":      ies.num_vertical,
            "num_horizontal":    ies.num_horizontal,
            "per_h": {
                str(h): {"peak_cd": v["peak"], "beam": v["beam"], "field": v["field"]}
                for h, v in metrics["per_h"].items()
            },
        }
    })


# ---------------------------------------------------------------------------
# GET /api/ies/polar?sid=&h_idx=0&scale=linear
# Return polar curve PNG as base64
# ---------------------------------------------------------------------------

@ies_viz_bp.route("/api/ies/polar", methods=["GET"])
def api_ies_polar():
    """
    Generate polar curve image for a loaded IES session.
    Query params:
      sid      — session id from /api/ies/load
      h_idx    — horizontal plane index (default 0)
      scale    — linear | sqrt | log (default linear)
      show_beam — true | false (default true)
    """
    sid = request.args.get("sid", "")
    if sid not in _IES_SESSIONS:
        return jsonify({"status": "error", "message": "Session not found. Call /api/ies/load first."})

    sess = _IES_SESSIONS[sid]
    from luxscale.ies_analyzer import plot_polar

    h_idx     = int(request.args.get("h_idx", 0))
    scale     = request.args.get("scale", "linear")
    show_beam = request.args.get("show_beam", "true") == "true"

    try:
        fig = plot_polar(sess["ies"], sess["metrics"], h_idx=h_idx,
                         scale=scale, show_beam=show_beam)
        return jsonify({"status": "success", "image": _fig_to_b64(fig)})
    except Exception as e:
        log_exception("api_ies_polar", e)
        return jsonify({"status": "error", "message": str(e)}), 500
```

```python
# ---------------------------------------------------------------------------
# GET /api/ies/panorama?sid=&room_h=3&room_sz=6&ct=warm
# Real room light simulation image
# ---------------------------------------------------------------------------

@ies_viz_bp.route("/api/ies/panorama", methods=["GET"])
def api_ies_panorama():
    """
    Ray-cast room simulation using actual IES candela data.
    Shows how the light actually looks in a room.

    Query params:
      sid       — session id
      room_h    — fixture/ceiling height in meters (default 3.0)
      room_sz   — room half-size in meters (default 6.0 → 12x12m room)
      ct        — color temperature: warm | neutral | cool (default warm)
      intensity — multiplier for brightness (default 1.0)
      w         — image width px (default 1024)
      h         — image height px (default 512)
    """
    import numpy as _np
    import math as _math
    from PIL import Image as _PILImage

    sid = request.args.get("sid", "")
    if sid not in _IES_SESSIONS:
        return jsonify({"status": "error", "message": "Session not found. Call /api/ies/load first."})

    sess = _IES_SESSIONS[sid]
    ies  = sess["ies"]

    W         = min(int(request.args.get("w",   1024)), 2048)
    PH        = min(int(request.args.get("h",    512)), 1024)
    room_h    = float(request.args.get("room_h",    3.0))
    room_sz   = float(request.args.get("room_sz",   6.0))
    intensity = float(request.args.get("intensity", 1.0))
    ct        = request.args.get("ct", "warm")

    try:
        va  = _np.array(ies.vertical_angles,   dtype=_np.float64)
        ha  = _np.array(ies.horizontal_angles, dtype=_np.float64)
        mat = _np.array([ies.candela_values[h] for h in ies.horizontal_angles], dtype=_np.float64)
        nH, nV  = mat.shape
        max_ha  = float(ha[-1])

        def lookup(v_flat, h_flat):
            v = _np.clip(v_flat.ravel(), va[0], va[-1])
```

```
    h = (_np.fmod(_np.fmod(h_flat.ravel(), 360.0) + 360.0, 360.0))
    if max_ha <= 90.5:
        h = _np.where(h > 270.0, 360.0 - h, h)
        h = _np.where(h > 180.0, h - 180.0,  h)
        h = _np.where(h >  90.0, 180.0 - h,  h)
    elif max_ha <= 180.5:
        h = _np.where(h > 180.0, 360.0 - h, h)
    h = _np.zeros_like(h) if nH == 1 else _np.clip(h, ha[0], ha[-1])
    with _np.errstate(divide='ignore', invalid='ignore'):
        hi  = _np.clip(_np.searchsorted(ha, h, 'right') - 1, 0, nH - 2)
        dha = _np.where(ha[hi+1] > ha[hi], ha[hi+1] - ha[hi], 1.0)
        ht  = _np.clip((h - ha[hi]) / dha, 0.0, 1.0)
        vi  = _np.clip(_np.searchsorted(va, v, 'right') - 1, 0, nV - 2)
        dva = _np.where(va[vi+1] > va[vi], va[vi+1] - va[vi], 1.0)
        vt  = _np.clip((v - va[vi]) / dva, 0.0, 1.0)
    hi1 = _np.minimum(hi + 1, nH - 1)
    vi1 = _np.minimum(vi + 1, nV - 1)
    c   = (  mat[hi,  vi ] * (1-vt) + mat[hi,  vi1] * vt) * (1-ht) \
        + (  mat[hi1, vi ] * (1-vt) + mat[hi1, vi1] * vt) * ht
    return c * intensity

cam_y  = 1.2
fix_y  = float(room_h)
ceil_y = fix_y + 0.5
s      = float(room_sz)
INF    = 1e30

lon = (_np.arange(W,  dtype=_np.float64) / W  - 0.5) * 2.0 * _math.pi
lat = (0.5 - _np.arange(PH, dtype=_np.float64) / PH) * _math.pi
LON, LAT = _np.meshgrid(lon, lat)
RX = _np.cos(LAT) * _np.sin(LON)
RY = _np.sin(LAT)
RZ = _np.cos(LAT) * _np.cos(LON)

t_hit = _np.full((PH, W), INF,  dtype=_np.float64)
NX = _np.zeros((PH, W), dtype=_np.float64)
NY = _np.zeros((PH, W), dtype=_np.float64)
NZ = _np.zeros((PH, W), dtype=_np.float64)

def hit_plane(Rcomp, O_val, p_val, nx, ny, nz, bounds_fn):
    with _np.errstate(divide='ignore', invalid='ignore'):
        t = _np.where(_np.abs(Rcomp) > 1e-9, (p_val - O_val) / Rcomp, INF)
    t = _np.where(t > 1e-4, t, INF)
    hx = RX * t;  hy = cam_y + RY * t;  hz = RZ * t
    t  = _np.where(bounds_fn(hx, hy, hz), t, INF)
    upd = t < t_hit
    _np.copyto(t_hit, t,  where=upd)
```

```
            _np.copyto(NX,    nx, where=upd)
            _np.copyto(NY,    ny, where=upd)
            _np.copyto(NZ,    nz, where=upd)

        hit_plane(RY, cam_y, 0.0,    0, 1, 0,  lambda hx,hy,hz: (_np.abs(hx)<=s) & (_np.abs(hz)<=s))
        hit_plane(RY, cam_y, ceil_y, 0,-1, 0,  lambda hx,hy,hz: (_np.abs(hx)<=s) & (_np.abs(hz)<=s))
        hit_plane(RX, 0.0,  -s,  1, 0, 0,      lambda hx,hy,hz: (hy>=0)&(hy<=ceil_y)&(_np.abs(hz)<=s))
        hit_plane(RX, 0.0,   s, -1, 0, 0,      lambda hx,hy,hz: (hy>=0)&(hy<=ceil_y)&(_np.abs(hz)<=s))
        hit_plane(RZ, 0.0,  -s,  0, 0, 1,      lambda hx,hy,hz: (hy>=0)&(hy<=ceil_y)&(_np.abs(hx)<=s))
        hit_plane(RZ, 0.0,   s,  0, 0,-1,      lambda hx,hy,hz: (hy>=0)&(hy<=ceil_y)&(_np.abs(hx)<=s))

        missed = t_hit >= INF * 0.5
        t_safe = _np.where(missed, 1.0, t_hit)
        HX = RX * t_safe;  HY = cam_y + RY * t_safe;  HZ = RZ * t_safe
        DX = HX;  DY = HY - fix_y;  DZ = HZ
        D2 = DX*DX + DY*DY + DZ*DZ
        D  = _np.sqrt(D2) + 1e-6
        V_ANG = _np.degrees(_np.arccos(_np.clip(-DY / D, -1.0, 1.0)))
        H_ANG = (_np.degrees(_np.arctan2(DZ, DX)) + 360.0) % 360.0
        CD  = lookup(V_ANG, H_ANG).reshape(PH, W)
        COS = _np.maximum(0.0, -(DX*NX + DY*NY + DZ*NZ) / D)
        LUX = CD * COS / (D2 + 1e-9)
        REFL = _np.where(NY > 0.5, 0.40, _np.where(NY < -0.5, 0.85, 0.72))
        RAD  = _np.where(missed, 0.0, LUX * REFL)

        valid = RAD[~missed]
        p99   = float(_np.percentile(valid, 99)) if valid.size > 0 else 1.0
        p99   = max(p99, 1e-4)
        T = _np.log1p(RAD / p99 * 10.0) / _math.log1p(10.0)
        T = _np.clip(T ** 0.68, 0.0, 1.0)

        CT = {"warm": (1.00, 0.84, 0.54), "neutral": (1.00, 0.96, 0.82), "cool": (0.82, 0.92, 1.00)}
        r, g, b = CT.get(ct, CT["warm"])
        img8 = (_np.stack([
            _np.clip(T * r + 0.012, 0, 1),
            _np.clip(T * g + 0.012, 0, 1),
            _np.clip(T * b + 0.012, 0, 1),
        ], axis=-1) * 255).astype(_np.uint8)

        pil = _PILImage.fromarray(img8, "RGB")
        buf = io.BytesIO()
        pil.save(buf, "PNG", optimize=True)
        buf.seek(0)
        return jsonify({"status": "success",
                        "image": base64.b64encode(buf.read()).decode(),
                        "width": W, "height": PH})
    except Exception as e:
```

```
        log_exception("api_ies_panorama", e)
        return jsonify({"status": "error", "message": str(e)}), 500


# ---------------------------------------------------------------------------
# GET /api/ies/plots/all?sid=   — return all 6 plots at once
# ---------------------------------------------------------------------------

@ies_viz_bp.route("/api/ies/plots/all", methods=["GET"])
def api_ies_all_plots():
    sid = request.args.get("sid", "")
    if sid not in _IES_SESSIONS:
        return jsonify({"status": "error", "message": "Session not found"}), 404

    sess = _IES_SESSIONS[sid]
    from luxscale.ies_analyzer import (
        plot_polar, plot_candela_profile, plot_heatmap,
        plot_beam_bar, plot_flux_curve
    )
    ies = sess["ies"]; m = sess["metrics"]
    try:
        return jsonify({
            "status": "success",
            "polar":    _fig_to_b64(plot_polar(ies, m, scale="linear")),
            "candela":  _fig_to_b64(plot_candela_profile(ies, m)),
            "heatmap":  _fig_to_b64(plot_heatmap(ies)),
            "beam_bar": _fig_to_b64(plot_beam_bar(ies, m)),
            "flux":     _fig_to_b64(plot_flux_curve(ies)),
        })
    except Exception as e:
        log_exception("api_ies_all_plots", e)
        return jsonify({"status": "error", "message": str(e)}), 500
```

Step 5 — Register the new routes in `app.py`

Open `LuxScaleAI/app.py` and add after the existing AI routes registration:

```
from luxscale.ies_routes import register_ies_routes
register_ies_routes(app)
```

Step 6 — Add “Polar Curve” button to `result.html`

Find the fixture result card in `result.html` where each result row is displayed. Add this button next to the existing action buttons:

```
<button class="btn-polar-curve"
        onclick="openPolarCurve('{{ luminaire_name }}', {{ power_w }})"
        title="View polar curve and room simulation">
  📊 Polar Curve
</button>
```

And add this JavaScript to `result.html`:

```
async function openPolarCurve(luminaireName, powerW) {
  // 1. Load the IES file and get a session
  const loadRes = await fetch('/api/ies/load', {
    method: 'POST',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify({luminaire_name: luminaireName, power_w: powerW})
  });
  const loadData = await loadRes.json();
  if (loadData.status !== 'success') {
    alert('Could not load IES file: ' + loadData.message);
    return;
  }

  const sid = loadData.session_id;
  const summary = loadData.summary;

  // 2. Open modal
  document.getElementById('polar-modal').style.display = 'flex';
  document.getElementById('polar-modal-title').textContent =
    luminaireName + ' — ' + powerW + 'W';

  // Show metrics
  document.getElementById('polar-metrics').innerHTML = `
    <div class="metric-grid">
      <div><label>Peak Candela</label><span>${summary.peak_candela?.toFixed(0)} cd</span></div>
      <div><label>Beam Angle (50%)</label><span>${summary.beam_angle?.toFixed(1) ?? 'n/a'}°</span></di
      <div><label>Field Angle (10%)</label><span>${summary.field_angle?.toFixed(1) ?? 'n/a'}°</span></
      <div><label>Integrated Lumens</label><span>${summary.total_lumens?.toFixed(0)} lm</span></div>
      <div><label>Declared Lumens</label><span>${summary.declared_lumens?.toFixed(0) ?? 'n/a'} lm</spa
      <div><label>LOR</label><span>${summary.lor_pct ?? 'n/a'}%</span></div>
      <div><label>Shape</label><span>${summary.shape}</span></div>
      <div><label>Symmetry</label><span>${summary.symmetry}</span></div>
    </div>
```

```
  `;

  // 3. Load polar curve
  document.getElementById('polar-img').src = '';
  document.getElementById('polar-loading').style.display = 'block';
  const polarRes = await fetch(`/api/ies/polar?sid=${sid}&scale=linear&show_beam=true`);
  const polarData = await polarRes.json();
  document.getElementById('polar-loading').style.display = 'none';
  if (polarData.status === 'success') {
    document.getElementById('polar-img').src = '' + polarData.image;
  }

  // 4. Load room simulation
  document.getElementById('panorama-loading').style.display = 'block';
  const panoRes = await fetch(`/api/ies/panorama?sid=${sid}&room_h=3&room_sz=6&ct=warm`);
  const panoData = await panoRes.json();
  document.getElementById('panorama-loading').style.display = 'none';
  if (panoData.status === 'success') {
    document.getElementById('panorama-img').src = '' + panoData.image;
  }

  // Store sid for scale/CT controls
  window._currentIESSid = sid;
}

// Scale change
document.querySelectorAll('.polar-scale-btn').forEach(btn => {
  btn.addEventListener('click', async () => {
    const scale = btn.dataset.scale;
    const res = await fetch(`/api/ies/polar?sid=${window._currentIESSid}&scale=${scale}&show_beam=true
    const data = await res.json();
    if (data.status === 'success')
      document.getElementById('polar-img').src = '' + data.image;
  });
});

// CT change
document.querySelectorAll('.ct-btn').forEach(btn => {
  btn.addEventListener('click', async () => {
    const ct = btn.dataset.ct;
    const roomH = document.getElementById('room-height-input').value || 3;
    const res = await fetch(
      `/api/ies/panorama?sid=${window._currentIESSid}&ct=${ct}&room_h=${roomH}`
    );
    const data = await res.json();
    if (data.status === 'success')
      document.getElementById('panorama-img').src = '' + data.image;
```

```
  });
});

// Close modal
document.getElementById('polar-modal-close').addEventListener('click', () => {
  document.getElementById('polar-modal').style.display = 'none';
});
```

And add the modal HTML to `result.html` (before closing `</body>`):

```
<div id="polar-modal" style="display:none; position:fixed; inset:0; background:rgba(0,0,0,0.85);
     z-index:9999; align-items:center; justify-content:center; padding:20px;">
  <div style="background:#1e1e1e; border-radius:12px; padding:24px; max-width:900px;
              width:100%; max-height:90vh; overflow-y:auto; color:#d0cec8;">

    <div style="display:flex; justify-content:space-between; align-items:center; margin-bottom:16px;">
      <h3 id="polar-modal-title" style="margin:0; color:#fff;">Polar Curve</h3>
      <button id="polar-modal-close"
              style="background:none; border:none; color:#888; font-size:24px; cursor:pointer;">✕</bu
    </div>

    <!-- Metrics grid -->
    <div id="polar-metrics" style="margin-bottom:16px;"></div>

    <!-- Polar scale controls -->
    <div style="margin-bottom:8px;">
      <span style="font-size:12px; color:#888; margin-right:8px;">Scale:</span>
      <button class="polar-scale-btn" data-scale="linear"
              style="padding:4px 10px; margin-right:4px; border-radius:4px;
                     background:#2a2a2a; border:1px solid #444; color:#d0cec8; cursor:pointer;">
        Linear
      </button>
      <button class="polar-scale-btn" data-scale="sqrt"
              style="padding:4px 10px; margin-right:4px; border-radius:4px;
                     background:#2a2a2a; border:1px solid #444; color:#d0cec8; cursor:pointer;">
        Sqrt
      </button>
      <button class="polar-scale-btn" data-scale="log"
              style="padding:4px 10px; border-radius:4px;
                     background:#2a2a2a; border:1px solid #444; color:#d0cec8; cursor:pointer;">
        Log
      </button>
    </div>

    <!-- Polar image -->
    <div style="text-align:center; margin-bottom:20px;">
```

```
      <div id="polar-loading" style="color:#888; padding:40px 0;">Loading polar curve…</div>
      <img id="polar-img" src="" style="max-width:100%; border-radius:8px;"
           onerror="this.style.display='none'">
    </div>

    <!-- Room simulation controls -->
    <h4 style="color:#aaa; margin-bottom:8px;">Room Light Simulation</h4>
    <div style="display:flex; gap:8px; align-items:center; margin-bottom:8px; flex-wrap:wrap;">
      <label style="font-size:12px; color:#888;">
        Room height (m):
        <input id="room-height-input" type="number" value="3" min="2" max="8" step="0.5"
               style="width:60px; background:#2a2a2a; border:1px solid #444;
                      color:#d0cec8; border-radius:4px; padding:2px 6px;">
      </label>
      <span style="font-size:12px; color:#888; margin-left:8px;">Color temp:</span>
      <button class="ct-btn" data-ct="warm"
              style="padding:4px 10px; border-radius:4px; background:#2a2a2a;
                     border:1px solid #444; color:#d0cec8; cursor:pointer;">
        Warm
      </button>
      <button class="ct-btn" data-ct="neutral"
              style="padding:4px 10px; border-radius:4px; background:#2a2a2a;
                     border:1px solid #444; color:#d0cec8; cursor:pointer;">
        Neutral
      </button>
      <button class="ct-btn" data-ct="cool"
              style="padding:4px 10px; border-radius:4px; background:#2a2a2a;
                     border:1px solid #444; color:#d0cec8; cursor:pointer;">
        Cool
      </button>
    </div>

    <div style="text-align:center;">
      <div id="panorama-loading" style="color:#888; padding:20px;">Loading room simulation…</div>
      <img id="panorama-img" src="" style="max-width:100%; border-radius:8px;"
           onerror="this.style.display='none'">
    </div>

  </div>
</div>
```

Summary: Files to Copy / Create / Edit

Action	File	What
COPY	`ieSControl/ies_analyzer.py` → `LuxScaleAI/luxscale/ies_analyzer.py`	Full IES engine
CREATE	`LuxScaleAI/luxscale/ies_routes.py`	New Flask routes (from Step 4 above)
EDIT	`LuxScaleAI/luxscale/ies_fixture_params.py`	Upgrade `ies_params_for_file()` (Step 3)
EDIT	`LuxScaleAI/app.py`	Add `register_ies_routes(app)` (Step 5)
EDIT	`LuxScaleAI/result.html`	Add Polar Curve button + modal (Step 6)
EDIT	`LuxScaleAI/requirements.txt`	Add `scipy`, `Pillow` (Step 2)

What Does NOT Change

- `uniformity_calculator.py` — keep it, it already does accurate point-by-point
- `lighting_calc/calculate.py` — keep it, only the lumen input improves
- All standards, fixture catalog, admin routes — untouched

Parameters Fed Into Calculations (Improved)

Parameter	Before	After
`lumens_per_lamp`	`header_lm × multiplier` (often wrong)	`estimate_lumens()` — integrated from candela data
`beam_angle_deg`	Min across H planes only	Global peak method (IES standard)
`field_angle_deg`	Not calculated	10% threshold crossing
`lor_pct`	Not calculated	Integrated / declared ratio
Polar curve	Not available	PNG via `/api/ies/polar`

Parameter	Before	After
Room simulation	Not available	Ray-cast panorama via `/api/ies/panorama`